MyCodeSchool

Exerciții — cifrele unui număr, cu șiruri de caractere

Recapitulare · C++ · cifre cu șiruri de caractere · 12 algoritmi, 60 probleme · David

`algoritmi-cifre-siruri.h` e doar de studiu. Are înăuntru algoritmii de mai jos, gata scriși, de data asta pe șir. Îl citești, îl închizi, apoi **scrii tu** algoritmul în headerul tău. Dacă te blochezi, îl trasezi pe hârtie, nu-l copiezi.

Cum lucrezi la fișă:

- **pentru fiecare algoritm îți faci headerul lui** — numele e scris în titlul fiecărei secțiuni;
- sus în header scrii **tu** algoritmul de bază, apoi cele 5 funcții, care îl folosesc;
- garda se scrie fără cratimă: `NUMAR_CIFRE_H_INCLUDED`, nu `NUMAR-CIFRE_H_INCLUDED`;
- numărul se citește ca șir, `cin >> s`, în `char s[256]`; **cifra** de pe indicele `i` este `s[i] - '0'`, iar **numărul de cifre** este `strlen(s)`;
- indicii se numără **de la stânga, începând de la 0**, în afară de cazul în care enunțul spune altfel;
- numerele sunt naturale (0 inclus), **fără zerouri la început** (0 se scrie `"0"`) și pot avea zeci de cifre: nu încap în `long long`;
- un număr **nou** construit în `rez` sau pe loc se scrie tot ca șir de cifre, fără zerouri la început; dacă nu rămâne nicio cifră, rezultatul este `"0"`;
- o funcție `void` care primește `s` îl **modifică pe loc**; la funcțiile cu `rez` rezultatul se scrie în `rez`; în rest funcția **întoarce** rezultatul, iar `cout` apare doar în `main`;
- în exemple, **ieșirea** este valoarea întoarsă sau șirul (`s`, `rez`) după apel;
- în `main.cpp` incluzi headerul la care lucrezi, apoi chemi funcțiile pe exemplele din fișă;
- lucrezi în proiectul tău `siruri-de-caractere` din Code::Blocks: Build and run (**F9**) înainte de fiecare commit.

Cum citești exemplele:

- un șir e scris între ghilimele, ca să se vadă zerourile și spațiile; **ghilimelele nu fac parte din șir**; o cifră sau un indice e scris simplu: `c = 2`;
- problemele marcate **doar cu șir** au în exemple numere care nu încap în `long long` (sau zerouri la început): cu `int` nu se pot rezolva;
- problemele **„Cele două variante"** cer să scrii și varianta cu `int`, ca să compari: `n` este numărul, `s` este același număr scris cu cifre, iar funcția întoarce `true` dacă variantele dau același rezultat.

Reguli:

- **Nu ai voie** să incluzi `algoritmi-cifre-siruri.h`, să chemi ceva din el sau să copiezi din el.
- **Ai voie** să chemi funcțiile **tale**, din headerele tale, oricâte, în orice combinație.
- Doar `char[]` și `<cstring>` (`strlen`, `strcpy`, `strcmp` ...).
- **Ștergerea și inserarea** se fac cu un șir auxiliar: `strcpy(aux, s + i + 1); strcpy(s + i, aux);`. `strcpy(s + i, s + i + 1)` nu e permis: sursa și destinația se suprapun.
- **Nu ai voie:** `string`, `vector` din bibliotecă, `#include <algorithm>`, `<cctype>`, `atoi`, `stoi`, `to_string`.
- Un vector de frecvență `int f[10]` ai voie începând cu secțiunea 8.

1. Numărul de cifre · `numar-cifre.h`

Studiezi din `algoritmi-cifre-siruri.h` : `nrCifre, cifraLa`

Care are mai multe cifre

doar cu șir 1.1

Scrie o funcție care întoarce 1 dacă `a` are mai multe cifre decât `b`, 2 dacă `b` are mai multe cifre, 0 dacă au tot atâtea.

`int celMaiLung(char a[], char b[])`

```
a = "4738" , b = "52"   ->  1
a = "7" , b = "12"      ->  2
a = "0" , b = "5"       ->  0
a = "12345678901234567890123456789" , b = "99999999999999999999999999999"
    ->  1
a = "90" , b = "31"     ->  0
```

Comparare de numere mari

doar cu șir 1.2

Scrie o funcție care compară două numere naturale date ca șiruri, fără zerouri la început: întoarce 1 dacă `a` > `b`, 2 dacă `b` > `a`, 0 dacă sunt egale. Numerele pot avea zeci de cifre.

`int comparaNumere(char a[], char b[])`

```
a = "4738" , b = "52"    ->  1
a = "999" , b = "1000"   ->  2
a = "12345678901234567890123456789" , b = "12345678901234567890123456788"
    ->  1
a = "500" , b = "500"    ->  0
a = "29999999999999999999999999999" , b = "30000000000000000000000000000"
    ->  2
a = "0" , b = "0"        ->  0
```

Cifra de la stânga

doar cu șir 1.3

Scrie o funcție care întoarce cifra de pe poziția `k`, numărată de la stânga începând de la 1. Dacă numărul nu are atâtea cifre, întoarce -1.

`int cifraDeLaStanga(char s[], int k)`

```
s = "18796" , k = 2                          ->  8
s = "18796" , k = 5                          ->  6
s = "18796" , k = 6                          ->  -1
s = "7" , k = 1                              ->  7
s = "0" , k = 1                              ->  0
s = "12345678901234567890123456789" , k = 25  ->  5
```

Cele două variante

1.4

Scrie și varianta cu int, cu `% 10` și `/ 10` : `int nrCifreInt(int n)` . Numărul `s` este `n` scris cu cifre. Funcția cerută întoarce `true` dacă `nrCifreInt(n)` și `nrCifre(s)` dau același rezultat.

`bool acelasiNumarDeCifre(int n, char s[])`

```
n = 4738 , s = "4738"           ->  true
n = 7 , s = "7"                 ->  true
n = 0 , s = "0"                 ->  true
n = 100000 , s = "100000"       ->  true
n = 999999999 , s = "999999999"  ->  true
```

Fără zerourile de la început

doar cu șir 1.5

Șirul poate începe cu zerouri, de exemplu `"00307"`. Scrie o funcție care întoarce câte cifre are numărul fără zerourile de la început; numărul 0 are o cifră.

`int cifreFaraZerouri(char s[])`

```
"00307"                              ->  3
"000"                                ->  1
"7"                                  ->  1
"1200"                               ->  4
"00000000000000000000000000000042"   ->  2
```

2. Suma cifrelor · `suma-cifre.h`

Studiezi din `algoritmi-cifre-siruri.h` : `sumaCifre`

Suma cifrelor pare

doar cu șir 2.1

Scrie o funcție care întoarce suma cifrelor pare ale numărului.

`int sumaCifrePare(char s[])`

```
"4738"                               ->  12
"1357"                               ->  0
"2000"                               ->  2
"0"                                  ->  0
"12345678901234567890123456890"      ->  60
```

Divizibil cu 3 sau cu 9

doar cu șir 2.2

Scrie o funcție care întoarce 9 dacă numărul se împarte la 9, 3 dacă se împarte la 3 dar nu la 9, 0 în rest. Folosește suma cifrelor, nu împărțirea numărului.

`int divizibilCu3Sau9(char s[])`

```
"123"                                ->  3
"333"                                ->  9
"1000"                               ->  0
"0"                                  ->  9
"12345678901234567890123456890"      ->  9
"99999999999999999999999999999"      ->  9
```

Divizibil cu 11

doar cu șir 2.3

Scrie o funcție care spune dacă numărul se împarte la 11, folosind suma alternată a cifrelor, de la dreapta: +, −, +, −... Nu împărți numărul.

`bool divizibilCu11(char s[])`

```
"121"                                ->  true
"918082"                             ->  true
"1234"                               ->  false
"0"                                  ->  true
"11111111111111111111111111111"      ->  true
"11111111111111111111111111111"      ->  false
```

Cele două variante

2.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `int sumaCifreInt(int n)`. Numărul `s` este `n` scris cu cifre. Funcția cerută întoarce `true` dacă `sumaCifreInt(n)` și `sumaCifre(s)` dau același rezultat.

`bool aceeasiSuma(int n, char s[])`

```
n = 4738 , s = "4738"              ->  true
n = 7 , s = "7"                    ->  true
n = 0 , s = "0"                    ->  true
n = 2000000000 , s = "2000000000"  ->  true
n = 999999999 , s = "999999999"    ->  true
```

Adunarea numerelor mari

doar cu șir 2.5

Scrie o funcție care pune în `rez` suma a două numere naturale date ca șiruri de cifre, fără zerouri la început, cu cel mult 100 de cifre. Adunarea se face cifră cu cifră, de la dreapta, cu transport.

`void adunaMari(char a[], char b[], char rez[])`

```
a = "999" , b = "1"  ->  "1000"
a = "0" , b = "0"    ->  "0"
a = "5" , b = "5"    ->  "10"
a = "123" , b = "9"  ->  "132"
a = "12345678901234567890123456" , b = "98765432109876543210987"
    ->  "133333332222333333322233332"
```

3. Produsul cifrelor · `produs-cifre.h`

Studiezi din `algoritmi-cifre-siruri.h`: `produsCifre`

Produsul cifrelor nenule

doar cu șir 3.1

Scrie o funcție care întoarce produsul cifrelor diferite de 0 ale numărului. Dacă numărul nu are nicio cifră nenulă, întoarce 0.

`long long produsCifreNenule(char s[])`

```
"405"                              ->  20
"1000"                             ->  1
"7"                                ->  7
"0"                                ->  0
"10000000000000000000000000002"    ->  2
"99999999999"                      ->  31381059609
```

Produs modulo m

doar cu șir 3.2

Scrie o funcție care întoarce produsul cifrelor numărului, modulo `m` ($1 \le m \le 1000$). Produsul întreg nu încape în `int`, așa că restul se ia pe parcurs.

`int produsCifreModulo(char s[], int m)`

```
s = "123456789" , m = 1000                          ->  880
s = "98765432198765432198765432198" , m = 1000      ->  0
s = "99999999999999999999999999999" , m = 7         ->  4
s = "0" , m = 5                                     ->  0
s = "5" , m = 1                                     ->  0
s = "11111111111111111111111111111" , m = 13       ->  1
```

Produsul de pe indicii pari

doar cu șir 3.3

Scrie o funcție care întoarce produsul cifrelor aflate pe indicii pari, numărați de la stânga începând de la 0 (prima cifră are indicele 0).

`long long produsIndicePar(char s[])`

```
"48723"                 ->  84
"55"                    ->  5
"7"                     ->  7
"0"                     ->  0
"123456789"             ->  945
"9191919191919191919"   ->  31381059609
```

Cele două variante

3.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `long long produsCifreInt(int n)`. Numărul `s` este `n` scris cu cifre. Funcția cerută întoarce `true` dacă `produsCifreInt(n)` și `produsCifre(s)` dau același rezultat.

`bool aceeasiProdus(int n, char s[])`

```
n = 405 , s = "405"             ->  true
n = 1000 , s = "1000"           ->  true
n = 7 , s = "7"                 ->  true
n = 0 , s = "0"                 ->  true
n = 99999999 , s = "99999999"   ->  true
n = 123456789 , s = "123456789" ->  true
```

Înmulțire cu o cifră

doar cu șir 3.5

Scrie o funcție care înmulțește pe loc numărul din `s` cu o cifră `c` ($0 \le c \le 9$). Rezultatul nu are zerouri la început; 0 înmulțit cu orice dă "0".

`void inmultesteCuCifra(char s[], int c)`

```
s = "12345678901234567890123456789" , c = 9
    ->  "111111110111111111011111111010"
s = "999" , c = 9   ->  "8991"
s = "5" , c = 2     ->  "10"
s = "123" , c = 0   ->  "0"
s = "0" , c = 7     ->  "0"
s = "1" , c = 1     ->  "1"
```

4. Cifra maximă și minimă · `cifra-maxima-minima.h`

Studiezi din `algoritmi-cifre-siruri.h`: `cifraMaxima, cifraMinima`

Cât de departe sunt

doar cu șir 4.1

Scrie o funcție care întoarce diferența dintre cifra maximă și cifra minimă a numărului.

`int diferentaMaxMin(char s[])`

```
"4738"                          ->  5
"4038"                          ->  8
"7"                             ->  0
"1111"                          ->  0
"12345678901234567890123456789" ->  9
```

Unde e maxima

doar cu șir 4.2

Scrie o funcție care întoarce indicele primei apariții a cifrei maxime, numărat de la stânga începând de la 0.

`int pozitiaMaximului(char s[])`

```
"4938"                  ->  1
"9189"                  ->  0
"9"                     ->  0
"0"                     ->  0
"12345678901234567890"  ->  8
```

Cifre crescătoare

doar cu șir 4.3

Scrie o funcție care spune dacă fiecare cifră este cel puțin egală cu cea dinaintea ei, adică cifrele nu scad niciodată de la stânga la dreapta.

`bool cifreCrescatoare(char s[])`

```
"1223"                           ->  true
"1232"                           ->  false
"7"                              ->  true
"0"                              ->  true
"11223344556677889999999999999"  ->  true
"11223344556677889999999999998"  ->  false
```

Cele două variante

4.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `int cifraMaximaInt(int n)` și `int cifraMinimaInt(int n)`. Numărul `s` este `n` scris cu cifre. Funcția cerută întoarce `true` dacă și maxima, și minima sunt aceleași în cele două variante.

`bool aceeasiMaximaMinima(int n, char s[])`

```
n = 4738 , s = "4738"  ->  true
n = 4038 , s = "4038"  ->  true
n = 7 , s = "7"        ->  true
n = 1111 , s = "1111"  ->  true
n = 0 , s = "0"        ->  true
```

Șterge o cifră, rămâne cel mai mare

doar cu șir 4.5

Scrie o funcție care șterge din număr, pe loc, o singură cifră, astfel încât numărul rămas să fie cel mai mare posibil. Dacă nu rămâne nicio cifră, `s` devine "0".

`void stergeUnaMaxim(char s[])`

```
"1432"                            ->  "432"
"4321"                            ->  "432"
"10"                              ->  "1"
"5"                               ->  "0"
"1099"                            ->  "199"
"12345678901234567890123456789"   ->  "23456789012345678901234567890"
"1000"                            ->  "100"
```

5. Prima cifră · `prima-cifra.h`

Studiezi din `algoritmi-cifre-siruri.h` : `primaCifra, ultimaCifra`

Prima și ultima

doar cu șir 5.1

Scrie o funcție care întoarce numărul de două cifre format din prima și ultima cifră a numărului.

`int primaSiUltima(char s[])`

```
"4738"                              ->  48
"120"                               ->  10
"7"                                 ->  77
"0"                                 ->  0
"12345678901234567890123456789O"    ->  10
```

Divizibil cu 2, 4, 5

doar cu șir 5.2

Scrie o funcție care întoarce câte dintre numerele 2, 4 și 5 divid numărul. Folosește doar ultimele cifre ale lui, nu împărțirea numărului.

`int divizoriDin245(char s[])`

```
"1100"                              ->  3
"12"                                ->  2
"7"                                 ->  0
"25"                                ->  1
"0"                                 ->  3
"12345678901234567890123456789O"    ->  2
"10000000000000000000000000000004"  ->  2
```

Începe și se termină la fel

doar cu șir 5.3

Scrie o funcție care spune dacă prima și ultima cifră a numărului sunt egale.

`bool primaEgalaCuUltima(char s[])`

```
"1331"                              ->  true
"1234"                              ->  false
"7"                                 ->  true
"0"                                 ->  true
"12345678901234567890123456789l"    ->  true
```

Cele două variante

5.4

Scrie și varianta cu int, cu `% 10` și `/ 10` : `int primaCifraInt(int n)` și `int ultimaCifraInt(int n)` . Numărul `s` este `n` scris cu cifre. Funcția cerută întoarce `true` dacă și prima, și ultima cifră sunt aceleași în cele două variante.

`bool aceleasiExtreme(int n, char s[])`

```
n = 4738 , s = "4738"               ->  true
n = 120 , s = "120"                 ->  true
n = 7 , s = "7"                     ->  true
n = 0 , s = "0"                     ->  true
n = 999999999 , s = "999999999"     ->  true
```

Prima cifră trece la coadă

doar cu șir 5.5

Scrie o funcție care mută pe loc prima cifră la sfârșitul șirului. Rezultatul rămâne șir de cifre, cu zerourile de la început: `"1000"` devine `"0001"`.

`void rotireStanga(char s[])`

```
"4738"                              ->  "7384"
"120"                               ->  "201"
"1000"                              ->  "0001"
"7"                                 ->  "7"
"0"                                 ->  "0"
"12345678901234567890123456890"     ->  "23456789012345678901234567890"
```

6. Oglinditul și palindromul · `oglindit-palindrom.h`

Studiezi din `algoritmi-cifre-siruri.h` : `oglindit, estePalindrom`

Lipește oglinditul

doar cu șir 6.1

Scrie o funcție care lipește pe loc, la dreapta lui `s` , cifrele lui în ordine inversă, ca să obții un palindrom cu număr par de cifre (`s` diferit de 0, cu cel mult 127 de cifre).

`void lipesteOglinditul(char s[])`

```
"123"                   ->  "123321"
"120"                   ->  "120021"
"7"                     ->  "77"
"10"                    ->  "1001"
"12345678901234567890"  ->  "1234567890123456789009876543210987654321"
```

Palindrom după o ștergere

doar cu șir 6.2

Scrie o funcție care spune dacă șirul devine palindrom după ce ștergi din el cel mult o cifră (se compară șirul rămas exact cum e, cu zerourile de la început).

`bool palindromFaraO(char s[])`

```
"1221"                                  ->  true
"1231"                                  ->  true
"1234"                                  ->  false
"1020"                                  ->  true
"7"                                     ->  true
"123456789012345678876543210987654321"  ->  true
"123456789012345678875654321098765432l"  ->  true
"123459967890123456788765432l098765432l"  ->  false
```

Perechi nepotrivite

doar cu șir 6.3

Scrie o funcție care întoarce câte perechi de cifre simetrice sunt diferite: prima cu ultima, a doua cu penultima și tot așa; cifra din mijloc nu face pereche.

`int perechiDiferite(char s[])`

```
"4738"                  ->  2
"1221"                  ->  0
"7"                     ->  0
"12345678900987654320"  ->  1
"1000"                  ->  1
```

Cele două variante

6.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `bool estePalindromInt(int n)`. Numărul `s` este `n` scris cu cifre. Funcția cerută întoarce `true` dacă `estePalindromInt(n)` și `estePalindrom(s)` dau același răspuns.

`bool aceeasiPalindrom(int n, char s[])`

```
n = 4738 , s = "4738"  ->  true
n = 120 , s = "120"    ->  true
n = 1221 , s = "1221"  ->  true
n = 7 , s = "7"        ->  true
n = 0 , s = "0"        ->  true
```

Adună cu oglinditul

doar cu șir 6.5

Scrie o funcție care adună repetat numărul cu oglinditul lui, până obține un palindrom, și întoarce în câți pași a ajuns; dacă numărul e deja palindrom, întoarce 0. Numerele de testat ajung la palindrom în cel mult 255 de cifre.

`int pasiPanaLaPalindrom(char s[])`

```
"87"                  ->  4
"56"                  ->  1
"121"                 ->  0
"0"                   ->  0
"89"                  ->  24
"1186060307891929990"  ->  261
```

7. Aparițiile unei cifre · `aparitii-cifra.h`

Studiezi din `algoritmi-cifre-siruri.h`: `contorAparitii, areCifra`

Prima apariție

doar cu șir 7.1

Scrie o funcție care întoarce indicele primei apariții a cifrei `c`, numărat de la stânga începând de la 0, sau -1 dacă `c` nu apare.

`int pozitiaPrimeiAparitii(char s[], int c)`

```
s = "12321" , c = 2                          ->  1
s = "12321" , c = 9                          ->  -1
s = "10200" , c = 2                          ->  2
s = "0" , c = 0                              ->  0
s = "12345678901234567890123456789" , c = 0  ->  9
```

Cel puțin de două ori

doar cu șir 7.2

Scrie o funcție care spune dacă cifra `c` apare în număr de cel puțin două ori.

`bool apareDeCelPutinDouaOri(char s[], int c)`

```
s = "12321" , c = 2                          ->  true
s = "12345" , c = 2                          ->  false
s = "1001" , c = 0                           ->  true
s = "0" , c = 0                              ->  false
s = "12345678901234567890123456789" , c = 5  ->  true
```

Între prima și ultima apariție

doar cu șir 7.3

Scrie o funcție care întoarce lungimea secvenței cuprinse între prima și ultima apariție a cifrei `c`, inclusiv ele; dacă `c` nu apare, întoarce 0.

`int lungimeaIntreAparitii(char s[], int c)`

```
s = "12321" , c = 2                          ->  3
s = "12321" , c = 1                          ->  5
s = "12321" , c = 3                          ->  1
s = "12321" , c = 9                          ->  0
s = "10000000000000000000000000001" , c = 0  ->  27
```

Cele două variante

7.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `int contorAparitiiInt(int n, int c)`. Numărul `s` este `n` scris cu cifre. Atenție la `n = 0`. Funcția cerută întoarce `true` dacă `contorAparitiiInt(n, c)` și `contorAparitii(s, c)` numără la fel.

`bool aceleasiAparitii(int n, char s[], int c)`

```
n = 12321 , s = "12321" , c = 2  ->  true
n = 12321 , s = "12321" , c = 9  ->  true
n = 10200 , s = "10200" , c = 0  ->  true
n = 0 , s = "0" , c = 0          ->  true
n = 100 , s = "100" , c = 0      ->  true
```

Cel mai lung bloc de cifre c

doar cu șir 7.5

Scrie o funcție care întoarce lungimea celui mai lung bloc de cifre `c` vecine; dacă `c` nu apare, întoarce 0.

`int celMaiLungBloc(char s[], int c)`

```
s = "1220003" , c = 0                          ->  3
s = "12321" , c = 2                            ->  1
s = "777" , c = 7                              ->  3
s = "0" , c = 0                                ->  1
s = "12321" , c = 9                            ->  0
s = "10000000000000000000000000002" , c = 0    ->  29
```

8. Frecvența cifrelor · `frecventa-cifre.h`

Studiezi din `algoritmi-cifre-siruri.h`: `frecventaCifre`

Prima cifră care lipsește

doar cu șir 8.1

Scrie o funcție care întoarce cea mai mică cifră care nu apare în număr, sau -1 dacă apar toate cele 10 cifre.

`int primaCifraLipsa(char s[])`

```
"4738"                          ->  0
"12345678901234567890123456789"  ->  -1
"1111"                          ->  0
"0"                             ->  1
"987654321"                     ->  0
```

Cifre care apar o singură dată

doar cu șir 8.2

Scrie o funcție care întoarce câte cifre diferite apar în număr exact o dată.

`int cifreUnice(char s[])`

```
"1213"                              ->  2
"1212"                              ->  0
"4738"                              ->  4
"7"                                 ->  1
"0"                                 ->  1
"12345678901234567890123456789999"  ->  0
```

Anagrame

doar cu șir 8.3

Scrie o funcție care primește două numere și spune dacă au exact aceleași cifre, fiecare apărând de același număr de ori.

`bool suntAnagrame(char a[], char b[])`

```
a = "12321" , b = "23121"  ->  true
a = "121" , b = "112"      ->  true
a = "12" , b = "11"        ->  false
a = "0" , b = "0"          ->  true
a = "12345678901234567890123456789" , b = "23456789012345678901234567890"
    ->  true
a = "12345678901234567890123456789" , b = "23456789012345678901234567902"
    ->  false
```

Cele două variante

8.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `void frecventaCifreInt(int n, int f[])`. Numărul `s` este `n` scris cu cifre. Atenție la `n = 0`. Funcția cerută întoarce `true` dacă cei doi vectori de frecvență (din `int` și din șir) sunt identici.

`bool aceeasiFrecventa(int n, char s[])`

```
n = 4738 , s = "4738"           ->  true
n = 1213 , s = "1213"           ->  true
n = 7777 , s = "7777"           ->  true
n = 0 , s = "0"                 ->  true
n = 999999999 , s = "999999999"  ->  true
```

Cel mai mare număr din cifrele lui

doar cu șir 8.5

Scrie o funcție care rearanjează pe loc cifrele șirului, ca să obții cel mai mare număr posibil cu ele.

`void celMaiMareDinCifre(char s[])`

```
"4738"                              ->  "8743"
"1203"                              ->  "3210"
"1000"                              ->  "1000"
"5"                                 ->  "5"
"0"                                 ->  "0"
"12345678901234567890123456789"     ->  "99988877766655544433322211000"
```

9. Eliminarea unei cifre · `eliminare-cifra.h`

Studiezi din `algoritmi-cifre-siruri.h` : `stergeCifra, taieZerouri, eliminaCifra`

Fără zerouri

doar cu șir 9.1

Scrie o funcție care șterge pe loc din număr toate cifrele 0. Rezultatul nu are zerouri la început, iar dacă nu rămâne nicio cifră, `s` devine "0".

`void faraZerouri(char s[])`

```
"10203"                          ->  "123"
"1000"                           ->  "1"
"4738"                           ->  "4738"
"0"                              ->  "0"
"12345678901234567890123456789"  ->  "123456789123456789123456789"
```

Cifra de pe un indice

doar cu șir 9.2

Scrie o funcție care șterge pe loc cifra de pe indicele `i`, numărat de la stânga începând de la 0. Dacă numărul nu are atâtea cifre, îl lasă neschimbat. După ștergere, zerourile de la început dispar: `"10307"` fără indicele 0 devine `"307"`.

`void eliminaDePePozitia(char s[], int i)`

```
s = "10307" , i = 0                          ->  "307"
s = "48723" , i = 2                          ->  "4823"
s = "48723" , i = 9                          ->  "48723"
s = "7" , i = 0                              ->  "0"
s = "3000000000000000000000000000001" , i = 0  ->  "1"
```

Cifra din mijloc

doar cu șir 9.3

Scrie o funcție care șterge pe loc cifra din mijloc (la număr impar de cifre) sau cele două cifre din mijloc (la număr par). Zerourile de la început dispar, iar dacă nu mai rămâne nimic, `s` devine "0".

`void stergeMijloc(char s[])`

```
"12345"                          ->  "1245"
"1234"                           ->  "14"
"7"                              ->  "0"
"10"                             ->  "0"
"102"                            ->  "12"
"12345678901234567890123456789"  ->  "123456789012347890123456789"
```

Cele două variante

9.4

Scrie și varianta cu int, cu `% 10` și `/ 10` : `int eliminaCifraInt(int n, int c)` . Numărul `s` este `n` scris cu cifre. Funcția cerută șterge cifra `c` dintr-o copie a lui `s` cu `eliminaCifra` , transformă șirul rămas în `int` , cifră cu cifră (`x = x * 10 + (t[i] - '0')`), și întoarce `true` dacă obține același număr ca `eliminaCifraInt(n, c)` .

`bool aceeasiEliminare(int n, char s[], int c)`

```
n = 10203 , s = "10203" , c = 0  ->  true
n = 72727 , s = "72727" , c = 7  ->  true
n = 4738 , s = "4738" , c = 5    ->  true
n = 777 , s = "777" , c = 7      ->  true
n = 0 , s = "0" , c = 0          ->  true
```

Fără duplicate

doar cu șir 9.5

Scrie o funcție care păstrează pe loc, din fiecare cifră, doar prima apariție, în ordinea inițială.

`void faraDuplicate(char s[])`

```
"1213"                                 ->  "123"
"7777"                                 ->  "7"
"0"                                    ->  "0"
"1001"                                 ->  "10"
"12345678901234567890123456789O"       ->  "1234567890"
"31415926535897932384626433832795"     ->  "314592687"
```

10. Inserarea și dublarea unei cifre · `inserare-cifra.h`

Studiezi din `algoritmi-cifre-siruri.h`: `insereaza, dublareCifra`

Dublează fiecare cifră

doar cu șir 10.1

Scrie o funcție care scrie pe loc fiecare cifră de două ori la rând (`s` diferit de 0, cu cel mult 127 de cifre).

`void dublareFiecare(char s[])`

```
"123"                  ->  "112233"
"105"                  ->  "110055"
"7"                    ->  "77"
"10"                   ->  "1100"
"12345678901234567890"  ->  "11223344556677889900112233445566778899001122334455667788990011223344556677889900"
```

După fiecare apariție

doar cu șir 10.2

Scrie o funcție care inserează pe loc cifra `c` imediat după fiecare apariție a cifrei `d`. Rezultatul se scrie fără zerouri la început ("05" devine "5").

`void inserareDupaFiecare(char s[], int d, int c)`

```
s = "1213" , d = 1 , c = 9  ->  "192193"
s = "555" , d = 5 , c = 0   ->  "505050"
s = "1234" , d = 7 , c = 9  ->  "1234"
s = "12" , d = 2 , c = 3    ->  "123"
s = "0" , d = 0 , c = 5     ->  "5"
s = "12345678901234567890123456789O" , d = 0 , c = 7
    ->  "12345678907123456789071234567890O7"
```

Puncte la mii

doar cu șir 10.3

Scrie o funcție care pune pe loc câte un punct din 3 în 3 cifre, începând de la dreapta: `"1234567"` devine `"1.234.567"`.

`void cuPuncteLaMii(char s[])`

```
"1234567"  ->  "1.234.567"
"123"      ->  "123"
"1000"     ->  "1.000"
"0"        ->  "0"
"1000000"  ->  "1.000.000"
"12345678901234567890123456789O"
    ->  "123.456.789.012.345.678.901.234.567.890"
```

Cele două variante

10.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `int inserareCifraInt(int n, int i, int c)`. Numărul `s` este `n` scris cu cifre. Aici `i` = câte cifre rămân în dreapta lui `c`. Funcția cerută inserează `c` într-o copie a lui `s`, ca să rămână `i` cifre în dreapta ei, transformă șirul în `int`, cifră cu cifră, și întoarce `true` dacă obține același număr ca `inserareCifraInt(n, i, c)`.

`bool aceeasiInserare(int n, char s[], int i, int c)`

```
n = 48723 , s = "48723" , i = 2 , c = 9  ->  true
n = 48723 , s = "48723" , i = 0 , c = 9  ->  true
n = 48723 , s = "48723" , i = 5 , c = 9  ->  true
n = 1357 , s = "1357" , i = 2 , c = 0    ->  true
n = 7 , s = "7" , i = 1 , c = 3          ->  true
```

Adună 1

doar cu șir 10.5

Scrie o funcție care adună 1, pe loc, la numărul din `s`: `"999"` devine `"1000"`.

`void aduna1(char s[])`

```
"999"                              ->  "1000"
"1299"                             ->  "1300"
"0"                                ->  "1"
"5"                                ->  "6"
"99999999999999999999999999999"    ->  "1000000000000000000000000000000"
"1234567890123456789"              ->  "1234567890123456790"
```

11. Cifra de control · `cifra-control.h`

Studiezi din `algoritmi-cifre-siruri.h`: `reducere, cifraControl`

Aceeași cifră de control

doar cu șir 11.1

Scrie o funcție care primește două numere și spune dacă au aceeași cifră de control.

`bool aceeasiCifraDeControl(char a[], char b[])`

```
a = "38" , b = "11"  ->  true
a = "12" , b = "13"  ->  false
a = "18" , b = "9"   ->  true
a = "0" , b = "9"    ->  false
a = "12345678901234567890123456789" , b = "99999999999999999999999999999"
    ->  true
```

Proba cu 9

doar cu șir 11.2

Scrie o funcție care primește `a`, `b` și `r` și spune dacă înmulțirea a · b = r trece proba cu 9: cifra de control a produsului cifrelor de control ale lui `a` și `b` trebuie să fie egală cu cifra de control a lui `r`.

`bool probaCuNoua(char a[], char b[], char r[])`

```
a = "23" , b = "14" , r = "322"                                  ->  true
a = "23" , b = "14" , r = "324"                                  ->  false
a = "0" , b = "5" , r = "0"                                      ->  true
a = "9" , b = "0" , r = "0"                                      ->  true
a = "1234567890" , b = "9876543210" , r = "12193263111263526900"  ->  true
a = "1234567890" , b = "9876543210" , r = "12193263111263526901"  ->  false
```

Restul la 9

doar cu șir 11.3

Scrie o funcție care întoarce restul împărțirii numărului la 9, fără să împarți numărul.

`int restLa9(char s[])`

```
"17"                                  ->  8
"4738"                                ->  4
"123456789"                           ->  0
"0"                                   ->  0
"12345678901234567890123456789 6"     ->  6
"99999999999999999999999999998"       ->  8
```

Cele două variante

11.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `int cifraControlInt(int n)`. Numărul `s` este `n` scris cu cifre. Funcția cerută întoarce `true` dacă `cifraControlInt(n)` și `cifraControl(s)` dau același rezultat.

`bool acelasiControl(int n, char s[])`

```
n = 9875 , s = "9875"             ->  true
n = 6 , s = "6"                   ->  true
n = 99 , s = "99"                 ->  true
n = 0 , s = "0"                   ->  true
n = 999999999 , s = "999999999"   ->  true
```

Cifra ștearsă

doar cu șir 11.5

Într-un număr a rămas o singură cifră ascunsă, scrisă `'?'`, iar numărul se împarte la 9. Scrie o funcție care întoarce cea mai mică cifră care pusă în locul lui `'?'` face numărul divizibil cu 9.

`int cifraLipsa(char s[])`

```
"12?4"                                ->  2
"?"                                   ->  0
"9?"                                  ->  0
"?5"                                  ->  4
"12345678901234567890?234567890"      ->  1
```

12. Numărul construit din cifre · `numar-din-cifre.h`

Studiezi din `algoritmi-cifre-siruri.h`: `cifrePare`

Mai mari decât ultima

doar cu șir 12.1

Scrie o funcție care pune în `rez` numărul format din cifrele strict mai mari decât ultima cifră a numărului, în ordinea inițială. Rezultatul nu are zerouri la început; dacă nu există nicio astfel de cifră, `rez` este "0".

`void maiMariDecatUltima(char s[], char rez[])`

```
"18796"                               ->  "879"
"4321"                                ->  "432"
"2222"                                ->  "0"
"7"                                   ->  "0"
"12345678901234567890123456789 0"     ->  "123456789123456789123456789"
```

Cifrele de pe indici pari

doar cu șir 12.2

Scrie o funcție care pune în `rez` numărul format din cifrele aflate pe indicii pari (0, 2, 4, ...), în ordinea inițială.

`void cifreIndicePar(char s[], char rez[])`

```
"48723"                              ->  "473"
"12"                                 ->  "1"
"7"                                  ->  "7"
"0"                                  ->  "0"
"10203"                              ->  "123"
"12345678901234567890123456789O"     ->  "135791357913579"
```

Jumătățile schimbate

doar cu șir 12.3

Scrie o funcție care schimbă pe loc între ele cele două jumătăți ale numărului; la număr impar de cifre, cifra din mijloc rămâne la mijloc. Zerourile de la început dispar.

`void jumatatiSchimbate(char s[])`

```
"123456"                             ->  "456123"
"12345"                              ->  "45312"
"102030"                             ->  "30102"
"7"                                  ->  "7"
"10"                                 ->  "1"
"12345678901234567890123456789O"     ->  "678901234567890123456789012345"
```

Cele două variante

12.4

Scrie și varianta cu int, cu `% 10` și `/ 10`: `int cifrePareInt(int n)`. Numărul `s` este `n` scris cu cifre. Funcția cerută construiește `rez` cu `cifrePare`, îl transformă în `int`, cifră cu cifră, și întoarce `true` dacă obține același număr ca `cifrePareInt(n)`.

`bool aceleasiCifrePare(int n, char s[])`

```
n = 4738 , s = "4738"     ->  true
n = 7 , s = "7"           ->  true
n = 0 , s = "0"           ->  true
n = 10203 , s = "10203"   ->  true
n = 1357 , s = "1357"     ->  true
```

Cel mai mic număr din cifrele lui

doar cu șir 12.5

Scrie o funcție care rearanjează pe loc cifrele șirului, ca să obții cel mai mic număr posibil cu ele, fără zero la început (dacă numărul are măcar o cifră nenulă).

`void celMaiMicDinCifre(char s[])`

```
"1203"                               ->  "1023"
"1000"                               ->  "1000"
"4738"                               ->  "3478"
"5"                                  ->  "5"
"0"                                  ->  "0"
"12345678901234567890123456789O"     ->  "100011222333444555666777888999"
```

Pentru fiecare funcție pe care o scrii, verific-o pe:

1. un număr cu **o singură cifră** și numărul **0**;
2. un număr cu **0 în interior** (4038) și unul cu **0 la coadă** (1200);
3. un număr cu **30 de cifre**;
4. cazul în care ce cauți **nu există** în număr;
5. rezultatul care ar începe cu **zerouri** sau ar rămâne **gol**.